

Nome e Cognome _____ Matricola _____

Corso di Calcolo Parallelo

Laurea Magistrale in Informatica—Università di Bologna

Prova scritta, 25 Maggio 2026

- La prova dura 90 minuti
- Durante la prova non è consentito consultare libri, appunti o altro materiale.
- Non è consentito l'uso di dispositivi elettronici (ad esempio, cellulari, tablet...), né interagire in alcun modo con gli altri ammessi alla prova pena l'esclusione, che potrà avvenire anche dopo il termine della stessa.
- Le risposte devono essere scritte **a penna** su questi fogli, in modo **leggibile**. Le parti illeggibili o scritte a matita saranno ignorate.
- Eventuali altri fogli possono essere utilizzati per la brutta copia, vanno consegnati ma non verranno valutati.
- I voti saranno pubblicati su AlmaEsami.
- I voti della prova scritta e del progetto restano validi fino alla sessione d'esame di **febbraio 2027** inclusa. Dopo tale data tutti i voti in sospeso saranno persi.

NON SCRIVERE NELLA TABELLA SOTTOSTANTE

D. 1	D. 2	D. 3	D. 4	D. 5
/ 6	/ 6	/ 6	/ 6	/ 6

Domanda 0.

Scrivere nome, cognome e numero di matricola in cima al primo foglio.

Domanda 1.

Dopo aver brevemente introdotto il concetto di architettura parallela a memoria distribuita, descriverne vantaggi e svantaggi motivando la risposta. Se lo si ritiene utile, è possibile fare riferimento ad esempi e/o applicazioni viste a lezione.

Risposta.

È importante ricordare che, ai fini di questo corso, “architettura parallela a memoria distribuita” è sinonimo di “supercomputer”; di conseguenza, le architetture parallele di nostro interesse:

- non sfruttano la rete Internet per collegare nodi l'uno con l'altro;
- hanno nodi di calcolo tipicamente omogenei, che possono eventualmente contenere CPU multicore e acceleratori;

Di conseguenza, risposte generiche che fanno riferimento implicito o esplicito ai sistemi distribuiti generici piuttosto che alle architetture parallele non possono essere considerate corrette.

Domanda 2.

Discutere se e come sia possibile parallelizzare il frammento di codice seguente usando OpenMP. Solo nel caso il codice possa essere parallelizzato, evidenziare eventuali aspetti positivi e negativi della soluzione proposta.

```
#define N ...
double A[N][N];

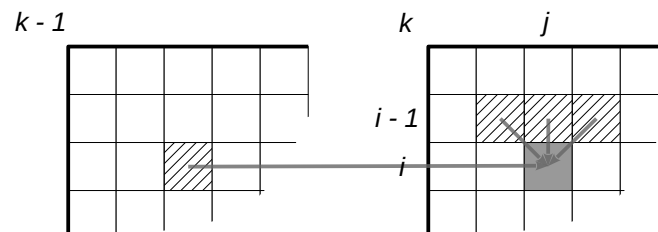
/* ... */

for (int k=0; k<500; k++) {
    for (int i=1; i<N; i++) {
        for (int j=1; j<N-1; j++) {
            A[i][j] = A[i][j] + (A[i-1][j-1] + A[i-1][j] + A[i-1][j+1])/3.0;
        }
    }
}
```

Risposta.

Consideriamo lo schema di accesso ai dati effettuato dal codice precedente. Il programma ripete per 500 volte una scansione della matrice A che avviene per riga. L'aggiornamento di A è di tipo read-modify-write: al valore precedente $A[i][j]$ viene sommato la media dei valori delle tre celle della riga precedente. Si presti attenzione che il codice non realizza una computazione di tipo *stencil*, perché viene modificata la stessa matrice che viene letta.

Le dipendenze possono essere illustrate nel modo seguente:



dove indichiamo in modo esplicito che il valore di $A[i][j]$ al termine dell'iterazione k dipende da quello al termine dell'iterazione $k - 1$.

Dalle considerazioni precedenti possiamo dedurre quanto segue:

- Il ciclo esterno sulla variabile k **non può** essere parallelizzato. Non è nemmeno possibile utilizzare la clausola `reduction()`: la clausola `reduction()` assume che i valori accumulati possano essere calcolati in modo indipendente, mentre quelli accumulati su $A[i][j]$ dipendono da valori della matrice modificata nelle iterazioni precedenti. (È possibile verificare sperimentalmente che il codice precedente non produce il risultato corretto se si parallelizza il ciclo esterno con `#pragma omp parallel for reduction(+:A[:N][:N])`).
- Il ciclo intermedio sulla variabile i **non può** essere parallelizzato, perché ci sono dipendenze tra una riga e la precedente, quindi righe diverse non possono essere calcolate in modo indipendente.
- Il ciclo interno sulla variabile j invece **può** essere parallelizzato: in altre parole, per ogni ite-

razione, fissata la riga i è possibile calcolarne tutti i nuovi valori in parallelo.

Riassumendo, il codice precedente può essere riscritto come:

```
for (int k=0; k<500; k++) {  
    for (int i=1; i<N; i++) {  
#pragma omp parallel for default(none) shared(k,i,A)  
        for (int j=1; j<N-1; j++) {  
            A[i][j] = A[i][j] + (A[i-1][j-1] + A[i-1][j] + A[i-1][j+1])/3.0;  
        }  
    }  
}
```

(le clausole default() e shared() potevano essere omesse). Questa soluzione potrebbe comportare un overhead considerevole, in base all'implementazione di OpenMP, a causa della ripetuta creazione e distruzione del team di thread. Per risolvere questo problema, si può creare il team una sola volta sola separando la direttiva parallel da for come discusso a lezione:

```
#pragma omp parallel  
for (int k=0; k<500; k++) {  
    for (int i=1; i<N; i++) {  
#pragma omp for  
        for (int j=1; j<N-1; j++) {  
            A[i][j] = A[i][j] + (A[i-1][j-1] + A[i-1][j] + A[i-1][j+1])/3.0;  
        }  
    }  
}
```

Domanda 3.

È dato un array $v[N]$ di lunghezza N contenente valori interi appartenenti all'insieme $\{0, \dots, 99\}$; l'array non è ordinato. Si vuole calcolare l'istogramma $h[100]$ del numero di occorrenze di ciascun valore in $v[]$. In altre parole, per ogni $k = 0, \dots, 99$, $h[k]$ deve essere il numero di volte in cui il valore k compare in $v[]$. Descrivere un algoritmo in grado di risolvere il problema su architetture a memoria distribuita usando MPI.

Si assuma che:

- la lunghezza N di $v[]$ sia multipla del numero P di processi MPI, e sia molto maggiore di P ;
- il valore di N sia inizialmente noto a tutti i processi;
- il contenuto di $v[]$ sia inizialmente noto solo al processo 0;
- al termine dell'esecuzione il contenuto dell'istogramma $h[]$ debba essere noto al processo 0.

Si descriva nel modo più preciso possibile il comportamento di ciascun processo MPI. **Non è richiesto scrivere codice:** è possibile usare pseudocodice e/o linguaggio naturale, purché **la spiegazione sia il più precisa possibile**. In particolare, si indichi quale/i funzioni MPI si userebbero nei vari passi, e quali buffer di memoria è necessario usare/allocare.

Risposta.

La soluzione può essere descritta dai passi seguenti:

1. Siano `my_rank` e `comm_sz` rispettivamente il rango del processo MPI corrente, e il numero di processi MPI.
2. Ogni processo alloca un array temporaneo `local_v[]` di `local_n = N/comm_sz` elementi.
3. Ogni processo alloca un array temporaneo `local_h[]` di 100 elementi; il contenuto di `local_h[]` deve essere inizializzato a zero.
4. L'array `v[]` viene distribuito tra tutti i processi: `MPI_Scatter(v, N/comm_sz, MPI_INT, local_v, N/comm_sz, MPI_INT, 0, MPI_COMM_WORLD)`.
5. Ciascun processo itera sul contenuto di `local_v[]` e aggiorna l'istogramma locale `local_h[]` (vedi sotto).
6. Viene effettuata una riduzione su `local_h[]`: `MPI_Reduce(local_h, h, 100, MPI_INT, MPI_SUM, 0, MPI_COMM_WORLD)`.

Per una soluzione completa e funzionante ad un problema molto simile a questo si veda `mpi_letters.c` nei *Parallel Etudes*.

Alcuni punti di attenzione:

- Non serve una sincronizzazione a barriera dopo il punto 5; le operazioni collettive non richiedono che tutti i processi le eseguano “contemporaneamente”.
- Non bisogna fare 100 riduzioni, una su ogni elemento di `local_h[]`: ne basta una sola.
- In alcuni casi è stato proposto il codice seguente per aggiornare gli istogrammi locali:

```
for (int k=0; k<100; k++) {  
    for (int i=0; i<local_n; i++) {
```

```
        if (local_v[i] == k)
            local_h[k]++;
    }
```

Questo approccio non è accettabile dato che risulta 100 volte più lento di quello corretto:

```
for (int i=0; i<local_n; i++) {
    local_h[local_v[i]]++;
}
```

- Attenzione che i parametri di `MPI_Scatter()` indicano il numeri di elementi inviati/ricevuti da ciascun processo, quindi devono essere entrambi `N/comm_sz`.

Domanda 4.

1. Descrivere un problema che, a proprio giudizio, **si presti** ad essere risolto in modo efficiente su una GPU; motivare la risposta.
2. Descrivere un problema che, a proprio giudizio, **non si presti** ad essere risolto in modo efficiente su una GPU; motivare la risposta.

È possibile fare riferimento ad esempi e/o applicazioni visti a lezione, oppure descriverne di nuovi.

Risposta.

Per il punto 1, ricordando che le GPU sono basate sul paradigma SIMD, si poteva indicare qualunque algoritmo che si presti a tale paradigma, es., somma di due array, prodotto matrice-vettore, computazioni stencil su immagini, eccetera. Molti di questi esempi sono stati visti o comunque discussi a lezione.

Il punto 2 è più complesso; la comunità scientifica ha prodotto versioni per GPU straordinariamente ingegnose anche per algoritmi che apparentemente non sono adatti all'approccio SIMD, ma non era richiesta (e nemmeno possibile!) una conoscenza dello stato dell'arte per cui qui ci si aspetta una qualunque risposta ragionevole.

Risposte ok: qualunque algoritmo di tipo task-parallel, come ad esempio il calcolo ricorsivo dei numeri di Fibonacci, oppure QuickSort o Merge Sort ricorsivi. L'approccio task-parallel in generale è difficile da rendere con una GPU. Per completezza segnalo che Merge Sort può essere realizzato in modo iterativo bottom-up; in questa forma diventa di tipo data-parallel e può essere realizzato da una GPU; tuttavia, questa variante di Merge Sort si studia raramente e pertanto assumo che non sia nota.

Risposte quasi ok:

- qualunque algoritmo preveda delle computazioni divergenti, dato che queste non sono realizzate in modo efficiente dalle GPU. Sebbene sia tecnicamente corretto che le GPU non forniscono prestazioni ottimali in caso di computazioni divergenti, questa risposta è un po' troppo generica: qualunque algoritmo poco più che banale esibisce un minimo di divergenza (vedi sotto l'esempio di Mandelbrot).
- algoritmi che richiedono frequenti trasferimenti di dati host-device e viceversa.
- algoritmi che richiedono sincronizzazioni globali a livello di griglia.

Risposte non ok:

- un algoritmo intrinsecamente seriale (questo tipo di algoritmi non è adatto ad alcuna architettura parallela, non solo alle GPU);
- algoritmi di ordinamento; abbiamo visto algoritmi (per quanto non work-efficient) che si prestano molto bene all'approccio data-parallel, come ad esempio odd-even transposition sort. Naturalmente esistono altri algoritmi (non visti a lezione) adatti alle GPU e più efficienti di odd-even transposition sort.

Il calcolo dell'insieme di Mandelbrot è stato considerato in alcuni casi come esempio del punto 1, e in altri come esempio del punto 2. (a causa di computazioni divergenti). Ho considerato corrette entrambe queste risposte, sebbene sia più ragionevole considerare il calcolo dell'insieme di Mandelbrot come computazione che può essere effettuata efficientemente su una GPU. È infatti vero che alcuni punti potrebbero richiedere meno iterazioni di altri per essere calcolati, ma è anche vero che

il numero di iterazioni ha un certo livello di località (in alcune aree, punti vicini richiedono circa lo stesso numero di iterazioni per essere calcolati) che di fatto rende il calcolo piuttosto efficiente sulle GPU. Nei *Parallel Etudes* c'è una versione GPU del calcolo dell'insieme di Mandelbrot; può essere istruttivo confrontarne le prestazioni con la quelle della versione OpenMP.

Domanda 5.

La funzione seguente usa un approccio divide-et-impera per calcolare il valore massimo presente nella porzione di un array $v[]$ compresa tra gli indici i e j , estremi inclusi.

```
int arraymax(const int *v, int i, int j)
{
    if (i > j)
        return INT_MIN; /* minimo valore rappresentabile dal tipo "int". */
    else if (i == j)
        return v[i];
    else {
        const int m = (i+j)/2;
        int l, r;
#pragma omp task shared(v, i, m, l)
        l = arraymax(v, i, m);
#pragma omp task shared(v, j, m, r)
        r = arraymax(v, m+1, j);
#pragma omp taskwait
        return (l > r ? l : r);
    }
}
```

La funzione viene inizialmente invocata da un singolo thread in una regione parallela con la chiamata $x = \text{arraymax}(v, 0, n-1)$.

Determinare *work*, *span* e *speedup* della funzione `arraymax()` applicata ad un array di lunghezza n , supponendo di avere $P = n$ processori.

Risposta.

Assumendo un numero illimitato di processori, il lavoro $Work(n)$ su un input di dimensione n soddisfa la seguente equazione di ricorrenza:

$$Work(n) = \begin{cases} 2 Work(n/2) + O(1) & \text{if } n > 1 \\ O(1) & \text{otherwise} \end{cases}$$

che ha soluzione $Work(n) = \Theta(n)$.

Lo span $Span(n)$ soddisfa la seguente equazione di ricorrenza:

$$Span(n) = \begin{cases} \max\{Span(n/2), Span(n/2)\} + O(1) & \text{if } n > 1 \\ O(1) & \text{otherwise} \end{cases}$$

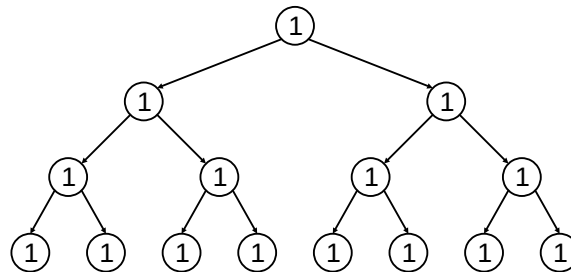
cioè la massima profondità dell'albero di ricorsione; tuttavia, dato che l'albero è perfettamente bilanciato, si poteva scrivere semplicemente:

$$Span(n) = \begin{cases} Span(n/2) + O(1) & \text{if } n > 1 \\ O(1) & \text{otherwise} \end{cases}$$

che ha soluzione $Span(n) = \Theta(\log n)$. Lo speedup è quindi $Work(n) / Span(n) = \Theta(n / \log n)$; lo

speedup ottimo che si può ottenere con $P = n$ processori è n , ma si vede che qui non può essere raggiunto.

Era anche possibile utilizzare un ragionamento di tipo grafico sull'albero di ricorsione per ottenere gli stessi risultati. Per evitare errori, è utile etichettare i nodi dell'albero di ricorsione con il lavoro svolto in quel nodo, anziché con la dimensione dell'array. Nel nostro caso l'albero ha l'aspetto seguente:



dove ci sono $\Theta(n)$ foglie, $\Theta(n)$ nodi e l'altezza è $\Theta(\log n)$; i numeri nei nodi indicano che ogni invocazione della funzione effettua lavoro aggiuntivo $O(1)$ oltre a quello svolto dalle due chiamate ricorsive.

Errori frequenti: errore nell'etichettatura dell'albero di ricorsione; risposte non motivate; risultati che non corrispondono ai calcoli che sono stati mostrati.